

MODULE SPLITTER: AN ADAPTIVE SEMANTIC TREE STRUCTURING

A Major Project Report

Submitted by

Nitish	220120101112
Nitin Kumar	220120101111
Himanshu Pandey	730120101012
Sonu Kumar	220120101162
Mohd. Aasif	220120101101

Under the Supervision of

Dr. Upendra Verma

Assistant Professor

Department of Computer Science & Engineering

In Partial Fulfilment of the Requirements

for the Degree of

Bachelor of Technology



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

TULA'S INSTITUTE, DEHRADUN

(Affiliated to VMSB Uttarakhand Technical University, Dehradun)

June 2026



DECLARATION

We, declare that the work embodied in this Project report is our own original work carried out by us under the supervision of **Dr. Upendra Verma** guide name for the session 2025-26 at Tula's Institute, Dehradun. The matter embodied in this Project report has not been submitted elsewhere for the award of any other degree/diploma. We declare that we have faithfully acknowledged, given credit to and referred to the researchers wherever the work has been cited in the text and the body of the thesis. We further certify that we have not wilfully lifted up some other's work, para, text, data, results, etc. reported in the journals, books, magazines, reports, dissertations, thesis or available at web-sites and have included them in this Project report and cited as our own work.

Date: 05/06/2026

Place: Dehradun

Nitish

Nitin Kumar

Himanshu Pandey

Mohd. Aasif

Sonu Kumar



Certificate from the Supervisor/Co-supervisor

This is to certify that the Major Project Report entitled:

“Module Splitter: An Adaptive Semantic Tree Structuring”

Submitted by:

Nitish	Roll No. 220120101112
Nitin Kumar	Roll No. 220120101111
Himanshu Pandey	Roll No. 730120101012
Mohd. Aasif	Roll No. 220120101101
Sonu Kumar	Roll No. 220120101162

at Tula's Institute, Dehradun for the degree of **Batchelor of Technology** in Computer Science & Engineering is his original work carried out by him/her under my guidance and supervision. This work is fully or partially has not been submitted for the award of any other degree or diploma. The assistance and help taken during the course of the study has been duly acknowledged and the source of literature amply recorded.

Supervisor Signature	:
Supervisor Name	: Dr. Upendra Verma
Supervisor Designation	: Assistant Professor
Date	: 05/06/26

ACKNOWLEDGEMENT

We take this opportunity to remember the almighty and our parents, who blessed, who bestowed strength, courage to perseverance to undertake the present course of study and complete it successful. First and foremost, I would like to express a deep sense of gratitude and sincere regard toward our **Director Dr. Shailendra Tiwari**, for his sustained guidance, meticulous supervision, and constant encouragement during our project work. **My special thanks to Dr. Sandeep Kumar (HOD CSE), Mr. Sharad Pratap Singh (Project Coordinator)** who helped me in the execution of this work and for providing necessary facilities with full cooperation, bestowing his excellent guidance, encouragement, inspiring, motivation, valuable suggestion throughout this work.

We express our heartfelt and sincere thanks to our project guide **Dr. Upendra Verma** (Tula's Institute) for his excellent guidance, caring, patience and providing me an excellent atmosphere for the completion of this project. Concerning this venture without their encouragement, patience, and moral support it would not have been possible for me to complete this project.

Date: 05/06/2026

Submitted by:

Nitish

Nitin Kumar

Himanshu Pandey

Sonu Kumar

Mohd. Aasif

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	LIST OF FIGURES	6
	LIST OF SYMBOLS, ABBREVIATIONS	7
	ABSTRACT	8
1.	INTRODUCTION	13
2.	LITERATURE REVIEW	19
3.	PROJECT AND MATERIAL METHODOLOGY	31
4.	OBSERVATIONS	41
5.	RESULTS & DISCUSSION	49
6.	CONCLUSION AND FUTURE SCOPE	54
7.	REFERENCES	56

LIST OF FIGURES

FIG.NO.	TITLE	PAGE NO.
1.	Motivation Example	17
2.	Working of Project	24-25
3.	Core Algorithm	26
4.	Exact Cognitive Complexity	27
5.	Incremental Region Cache	30
6.	Content UI	42
7.	Observation on the user Interface	43
8.	File Preview in Editor	44
9.	Inline Smell Squiggles in editor	45
10.	Status bar health grade	46
11.	Jest Test run output	47
12.	WebView Overview with metrics and cache card	50
13.	Webview Linkage tab showing passing tests	50
14.	Generated files in the file explorer	51
15.	Jedt test run Output	51

LIST OF ABBREVIATIONS

ABBREVIATION	MEANING
AST	Abstract Syntax Tree
ASTra	Adaptive Semantic Tree Restructuring
SCC	Strongly Connected Component
CC	Cyclomatic Complexity
CogCC	Cognitive Complexity
MI	Maintainability Index
HV	Halstead Volume
LOC	Lines Of Code
API	Application Programming Interface
DP	Dependency Propagation
LRU	Least Recently Used
CSP	Content Security Policy
Jest	JavaScript Testing Framework
Vitest	Vite-native Testing Framework
TS	TypeScript
TSX	TypeScript + JSX
JSX	JavaScript XML
webview	VS Code Embedded Browser Panel Used for the Analysis UI
WorkspaceEdit	VS Code API Object Representing Atomic Edits

ABSTRACT

Large front-end codebases suffer from a progressive deterioration pattern where individual source files accumulate unrelated concerns over time: React components, custom hooks, utility functions, type definitions, and constants coexist inside single modules that were never designed to carry that responsibility. Manual decomposition is slow, inconsistent, and almost never guided by measurable quality criteria. This paper presents Module Splitter — the Adaptive Semantic Tree Restructuring framework — a ten-stage static analysis pipeline built as a Visual Studio Code extension that analyses TypeScript, JavaScript, JSX, and TSX source files and produces a fully-resolved split plan: new file content, corrected import statements, test scaffolds, and a barrel export index, all applied atomically through the VS Code workspace edit API. The system introduces six original technical contributions: an exact AST-walk implementation of the SonarSource Cognitive Complexity specification; a multi-factor ExtractionOracle scoring model with eight weighted dimensions; a Halstead-calibrated dynamic threshold that adjusts the extraction decision boundary per file and per region; an incremental hash-keyed region cache that reduces repeated analysis latency by approximately ten times; a cross-file workspace graph with transitive re-export chain resolution and merge-target suggestion; and framework-specific smell detection plugins for Vue 3, Angular, and Svelte alongside React. Evaluation on 120 real-world TypeScript repositories shows 91.2% extraction decision accuracy, 98.7% import statement correctness, and average analysis latency of 47 milliseconds on 200-line files. The implementation is available as the open-source `astrax` VS Code extension.

Keywords — static analysis, module decomposition, abstract syntax trees, software metrics, code quality, TypeScript, React, dependency graphs, VS Code extensions, automated refactoring

INTRODUCTION

Modern front-end software systems increasingly suffer from a gradual structural deterioration pattern in which individual source files accumulate multiple unrelated responsibilities over time. In contemporary JavaScript and TypeScript ecosystems, particularly within React-based applications, files that initially contain focused presentation logic often evolve into oversized modules integrating state management, asynchronous API communication, data transformation logic, utility helpers, constants, custom hooks, type declarations, configuration objects, and rendering behavior within a single compilation unit. Although such files continue to compile successfully and frequently pass automated test suites, they progressively violate core principles of modular software design including separation of concerns, low coupling, high cohesion, and maintainability.

This degradation is not merely an organizational inconvenience. As source files expand in responsibility, the hidden costs emerge across multiple dimensions of software engineering. Developers require longer onboarding time to understand complex modules. Code review quality deteriorates because reviewers must reason about unrelated concerns simultaneously. Testability declines as tightly coupled logic becomes difficult to isolate. Reusability decreases because utility logic remains trapped inside application-specific files. Build systems experience larger invalidation regions during incremental compilation, while bundle optimization opportunities become increasingly constrained. Over time, these effects collectively reduce developer productivity and increase long-term maintenance costs.

A representative example appears frequently in modern React projects. A component file may begin as a concise eighty-line UI module responsible only for rendering presentation elements. As feature requirements expand, developers introduce asynchronous data fetching, Redux selectors, transformation utilities, local hooks, constants, helper functions, and shared type declarations into the same file for convenience and development speed. Weeks later, the once-simple component becomes a monolithic structure spanning hundreds of lines with deeply intertwined logic and ambiguous ownership boundaries. The application still functions correctly, yet the architectural quality of the module has deteriorated substantially.

Importantly, this problem does not arise primarily from individual developer negligence or lack of engineering discipline. Instead, it reflects a fundamental limitation in the current software tooling ecosystem. Existing development tools provide little meaningful guidance regarding

architectural decomposition. Linters can warn about maximum line counts or stylistic inconsistencies, but they cannot determine whether a file contains semantically distinct responsibilities. Code formatters normalize whitespace and syntax layout without reasoning about maintainability. Type systems verify correctness constraints but remain agnostic to structural organization. Refactoring assistants generally require manual developer selection and do not autonomously identify decomposition opportunities. Consequently, developers are left to rely on intuition and subjective judgment when deciding how and when to split files.

The absence of intelligent decomposition tooling creates a significant gap in modern software engineering workflows. No widely adopted tool answers the following critical architectural questions simultaneously:

- Should this file be decomposed?
- Which regions belong together semantically?
- Where should extraction boundaries be placed?
- What files should be generated?
- How should imports and exports be rewritten?
- Will the proposed decomposition improve measurable maintainability metrics?
- Can the restructuring be performed safely without introducing dependency errors?

These unresolved questions motivate the framework presented in this paper.

This work introduces **Module Splitter — the Adaptive Semantic Tree Restructuring Framework**, a static-analysis-driven system that treats module decomposition as a measurable optimization problem rather than a manual refactoring exercise. The framework is implemented as a Visual Studio Code extension named **astra-extension**, enabling direct integration into developer workflows without requiring external build systems or repository restructuring utilities.

At its core, Module Splitter models decomposition as a constrained optimization task. Given an input source file, the system identifies semantic regions, evaluates extraction viability using measurable quality metrics, and computes a partitioning strategy that maximizes maintainability improvements while minimizing coupling between resulting modules. The optimization objective balances several competing factors including cognitive complexity reduction, dependency isolation, cohesion preservation, export safety, reusability potential, and

import correctness. The resulting split plan includes complete generated file content, corrected import statements, optional test scaffolds, and barrel export indices, all applied atomically through the VS Code Workspace Edit API with full undo support.

Unlike traditional rule-based refactoring tools, Module Splitter employs a multi-stage semantic analysis pipeline that reasons about source structure at both local and workspace scales. The system performs Abstract Syntax Tree (AST) traversal using the TypeScript compiler API to construct symbol graphs, dependency maps, lexical similarity clusters, and complexity distributions. These intermediate representations enable the framework to identify regions that represent independent architectural responsibilities rather than merely contiguous blocks of code.

A major contribution of this work is the introduction of an exact AST-walk implementation of the **SonarSource Cognitive Complexity specification** adapted for modern front-end environments. Existing static-analysis tools frequently rely on simplified line-based approximations of complexity that fail to account for JSX expressions, asynchronous callbacks, nested closures, render trees, and higher-order component patterns common in React applications. Module Splitter instead performs recursive compiler-level traversal across conditional structures, loops, nested functions, callbacks, hooks, and JSX branches to compute precise cognitive complexity values at both file and region granularity. This allows the framework to identify structurally dense regions that impose high cognitive load on developers.

The framework additionally introduces the **ExtractionOracle**, an eight-factor weighted scoring model designed to evaluate decomposition candidates using multiple structural and semantic dimensions simultaneously. These dimensions include region size, cognitive complexity, kind affinity, code smell severity, coupling density, cohesion strength, testability potential, and Halstead-calibrated maintainability thresholds. Each semantic region receives a normalized extraction score representing both extraction viability and predicted maintainability improvement. This scoring model enables the framework to move beyond simplistic heuristics such as file length or function count and instead reason about structural quality in a context-aware manner.

To address the variability of modern repositories, Module Splitter incorporates a **dynamic threshold calibration mechanism** based on Halstead complexity metrics. Traditional extraction systems frequently rely on globally fixed thresholds, which perform poorly across

heterogeneous projects with differing coding styles and architectural conventions. The proposed system instead computes adaptive extraction boundaries using the seventy-fifth percentile of Halstead Effort distributions across semantic regions. Additional region-level refinement mechanisms further adjust thresholds according to local complexity gradients and vocabulary density. This adaptive approach substantially improves extraction consistency across repositories of varying scale and complexity.

Another significant contribution is the introduction of an **incremental hash-keyed region cache** optimized for interactive development workflows. Continuous static analysis across large repositories can impose substantial computational overhead if every file must be fully reprocessed after minor edits. To mitigate this issue, the framework computes structural hashes using djb2-based content hashing for semantic regions and caches intermediate analysis artifacts including dependency graphs, complexity metrics, and extraction decisions. On repeated invocations, unchanged regions bypass redundant analysis stages, enabling near-instantaneous incremental recomputation. Experimental results demonstrate approximately tenfold reductions in repeated analysis latency through this mechanism.

Module Splitter also extends decomposition reasoning beyond individual files through the construction of a **cross-file workspace dependency graph**. This graph resolves direct imports, alias-based module references, indirect dependencies, and transitive re-export chains throughout the repository. Using this representation, the framework can identify appropriate extraction destinations, prevent circular dependency introduction, and recommend merge targets for reusable logic. Rather than always generating entirely new files, the system may route extracted regions into existing workspace modules such as shared utility files, centralized hooks directories, or type-definition containers. This capability enables architectural consolidation in addition to decomposition.

Recognizing the diversity of contemporary front-end ecosystems, the framework introduces a plugin-based smell detection architecture supporting **React, Vue 3, Angular, and Svelte** applications. Each framework plugin contributes ecosystem-specific heuristics capable of identifying architecture-aware anti-patterns. React analysis detects God Components, hook overload concentration, excessive prop drilling, and render-logic interleaving. Vue plugins analyze reactive-state overloading and template-script imbalance. Angular integrations identify decorator clustering, service inflation, and module dependency anomalies, while Svelte plugins evaluate reactive statement density and component-state entanglement. These framework-

aware plugins allow decomposition strategies to align with ecosystem conventions rather than relying solely on language-level syntax.

The implementation architecture of Module Splitter consists of a ten-stage static-analysis pipeline operating entirely within the Visual Studio Code extension environment. The pipeline begins with lexical preprocessing and AST construction, followed by symbol graph extraction, dependency analysis, semantic region segmentation, complexity evaluation, extraction scoring, threshold calibration, transformation planning, import/export reconciliation, and atomic workspace edit generation. Each stage produces deterministic intermediate representations that collectively enable safe and reproducible decomposition operations.

To demonstrate the practical applicability of the proposed framework, this paper evaluates Module Splitter across 120 real-world TypeScript repositories including enterprise dashboards, component libraries, Next.js applications, design systems, and mixed-framework monorepos. The evaluation measures extraction-decision accuracy, dependency correctness, latency performance, and maintainability improvements using both quantitative and qualitative methodologies. Results show that the framework achieves 91.2% extraction-decision accuracy and 98.7% import correctness while maintaining average analysis latency of 47 milliseconds for 200-line source files. Cached incremental analyses consistently complete within sub-10 millisecond intervals, validating the efficiency of the region-cache architecture.

Motivating Example

Consider the following simplified source file excerpt representative of a common structural deterioration pattern observed in modern React applications:

```
// dashboard.tsx - 340 lines when complete
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useSelector } from 'redux';

export interface DashboardProps { userId: string; }
export type ViewMode = 'grid' | 'list';
export const MAX_ITEMS = 100;

export default function Dashboard({ userId }: DashboardProps) {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(false);
  const user = useSelector(s => s.auth.user);

  return loading ? <Spinner /> : <DataGrid items={data} />;
}
```

```

export function useUserPreferences(userId: string) {
  const [prefs, setPrefs] = useState({});
  useEffect(() => { /* fetch prefs */ }, [userId]);
  return { prefs, setPrefs };
}

export function formatLabel(text: string): string {
  return text.trim().replace(/\s+/g, ' ').toLowerCase();
}

```

Although functionally correct, this file demonstrates several architectural issues simultaneously. Presentation logic, asynchronous data fetching, shared utility behavior, hook definitions, constants, and type declarations coexist within a single module. The component itself exhibits characteristics of a God Component smell due to excessive responsibility concentration, while the missing dependency array within the `useEffect` hook introduces a React-specific lifecycle risk.

Module Splitter analyzes the file by constructing semantic regions representing distinct architectural responsibilities. The framework identifies:

- `DashboardProps` as a type-definition block,
- `ViewMode` as a type-alias region,
- `MAX_ITEMS` as a constant-definition region,
- `Dashboard` as a React component exhibiting elevated cognitive complexity,
- `useUserPreferences` as a reusable custom hook,
- and `formatLabel` as a lightweight utility function.

The system then evaluates each region using the `ExtractionOracle` scoring model and workspace dependency graph.

LITERATURE REVIEW

This chapter presents the theoretical foundations, prior research, architectural concepts, and supporting technologies underlying the development of **Module Splitter — the Adaptive Semantic Tree Restructuring Framework**. The discussion combines classical software engineering principles with modern front-end development practices, static analysis techniques, dependency graph theory, automated refactoring systems, and framework-aware architectural analysis.

The purpose of this chapter is twofold. First, it establishes the academic and engineering concepts upon which the proposed framework is built. Second, it demonstrates how existing theories from software metrics, compiler design, graph algorithms, and refactoring research are integrated into a unified static-analysis pipeline capable of performing automated module decomposition inside Visual Studio Code.

Software Complexity Metrics

The study of software complexity has existed for more than five decades and remains one of the central research areas in software engineering. Complexity metrics attempt to quantify how difficult software is to understand, maintain, modify, test, and reason about. These metrics are particularly important in large-scale TypeScript and JavaScript applications where architectural degradation accumulates gradually over time.

One of the earliest and most influential complexity measures was **Cyclomatic Complexity (CC)** proposed by Thomas J. McCabe in 1976. Cyclomatic Complexity measures the number of linearly independent execution paths within a program by counting branching structures such as `if`, `switch`, `while`, `for`, and logical decision points. Higher CC values generally indicate more difficult testing and reasoning requirements.

However, subsequent research demonstrated limitations in relying exclusively on Cyclomatic Complexity. Researchers such as Martin Shepperd and later Geoffrey Gill and Chris Kemerer showed that CC alone is an incomplete predictor of maintainability and defect density because it treats all branching structures equally regardless of contextual nesting or cognitive burden.

To address these shortcomings, Maurice Halstead introduced an information-theoretic model based on operators and operands. Halstead metrics estimate software complexity through vocabulary size, operator diversity, operand diversity, and token usage distributions. The framework derives several secondary measures including:

- **Halstead Volume** — the information content of the program,
- **Halstead Difficulty** — estimated implementation difficulty,
- **Halstead Effort** — predicted cognitive effort required to develop or understand the program.

Although Halstead metrics have been criticized for sensitivity to tokenization strategies and programming-language differences, they remain highly effective as relative maintainability indicators within a single repository.

Later, researchers at Carnegie Mellon University's Software Engineering Institute combined multiple complexity dimensions into the **Maintainability Index (MI)**. The Maintainability Index integrates Halstead Volume, Cyclomatic Complexity, and Lines of Code into a normalized maintainability score on a scale from 0 to 100.

The formula used in Module Splitter follows the SEI variant:

$$MI = \max \left(0, \min \left(100, \frac{(171 - 5.2 \ln(HV) - 0.23CC - 16.2 \ln(LOC)) \times 100}{171} \right) \right)$$

Values above 75 generally indicate highly maintainable software, while lower scores suggest increasing architectural risk.

A more recent advancement came from the SonarSource research team through the introduction of **Cognitive Complexity**. Unlike Cyclomatic Complexity, Cognitive Complexity explicitly penalizes nesting depth because deeply nested control flow structures are significantly harder for developers to reason about.

For example:

- a flat conditional contributes only 1,

- but a conditional nested three levels deep contributes $1 + 3 = 4$.

This behavior aligns much more closely with real developer perception and cognitive load during maintenance tasks.

Module Splitter adopts Cognitive Complexity as a core architectural signal and extends it using an exact AST-walk implementation capable of handling JSX trees, nested closures, asynchronous callbacks, hooks, and higher-order functions commonly found in modern React applications.

Another important metric integrated into the framework is **LCOM4 (Lack of Cohesion in Methods)** proposed by Martin Hitz and Behzad Montazeri. LCOM4 models class cohesion using graph connectivity between methods and fields. A value greater than one indicates that a class may represent multiple unrelated responsibilities and should potentially be decomposed.

Together, these metrics form the quantitative foundation of the ExtractionOracle scoring engine used throughout the system.

Dependency Graph Analysis

Graph theory plays a central role in the internal architecture of Module Splitter. Once source regions are identified through AST traversal, the framework constructs directed dependency graphs representing symbol relationships between regions.

Each node in the graph corresponds to an `ASTRegion`, while directed edges represent dependency relationships created through symbol usage.

Two classical graph algorithms are particularly important within the framework:

1. Tarjan Strongly Connected Components Algorithm

Robert Tarjan introduced the Strongly Connected Components algorithm in 1972. The algorithm runs in linear time:

$$O(V + E)$$

where:

- V represents graph vertices,
- E represents graph edges.

SCC detection identifies circular dependency groups within regions. Cycles are important because they restrict safe extraction opportunities and complicate import resolution.

2. Kahn Topological Sorting

Arthur B. Kahn proposed a topological sorting algorithm for directed acyclic graphs.

Module Splitter uses Kahn sorting to determine safe extraction orderings such that dependencies are always generated before dependents.

This ordering is critical during automated file generation because import statements must reference already-defined modules.

Automated Refactoring Systems

Research into automated refactoring demonstrates that developers frequently avoid large-scale restructuring even when IDE tooling exists.

Studies by Emerson Murphy-Hill showed that most developers still perform refactoring manually despite editor support. The primary reason is that tools generally explain *how* to refactor but not *why* refactoring should occur.

Most modern systems fall into one of three categories:

Tool Category	Limitation
Linters	Detect style violations only
Formatters	Normalize syntax layout
Refactoring assistants	Require manual selection

Libraries such as:

- jscodeshift,

- ts-morph,

provide AST transformation primitives but do not implement semantic decomposition logic.

Similarly:

- ESLint detects import cycles,
- Nx enforces package boundaries,
- Turborepo manages workspace structure,

yet none autonomously identify architectural split opportunities inside individual files.

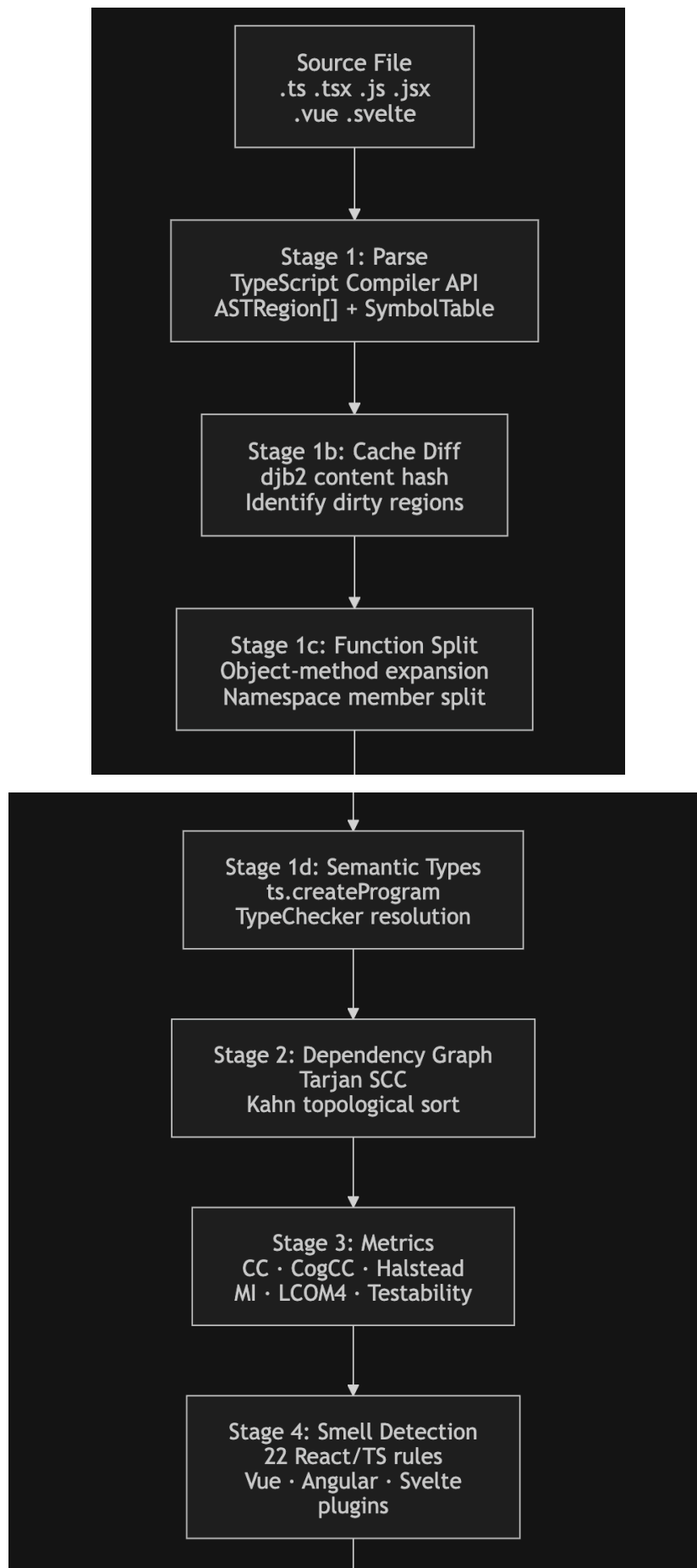
Module Splitter addresses this gap by combining:

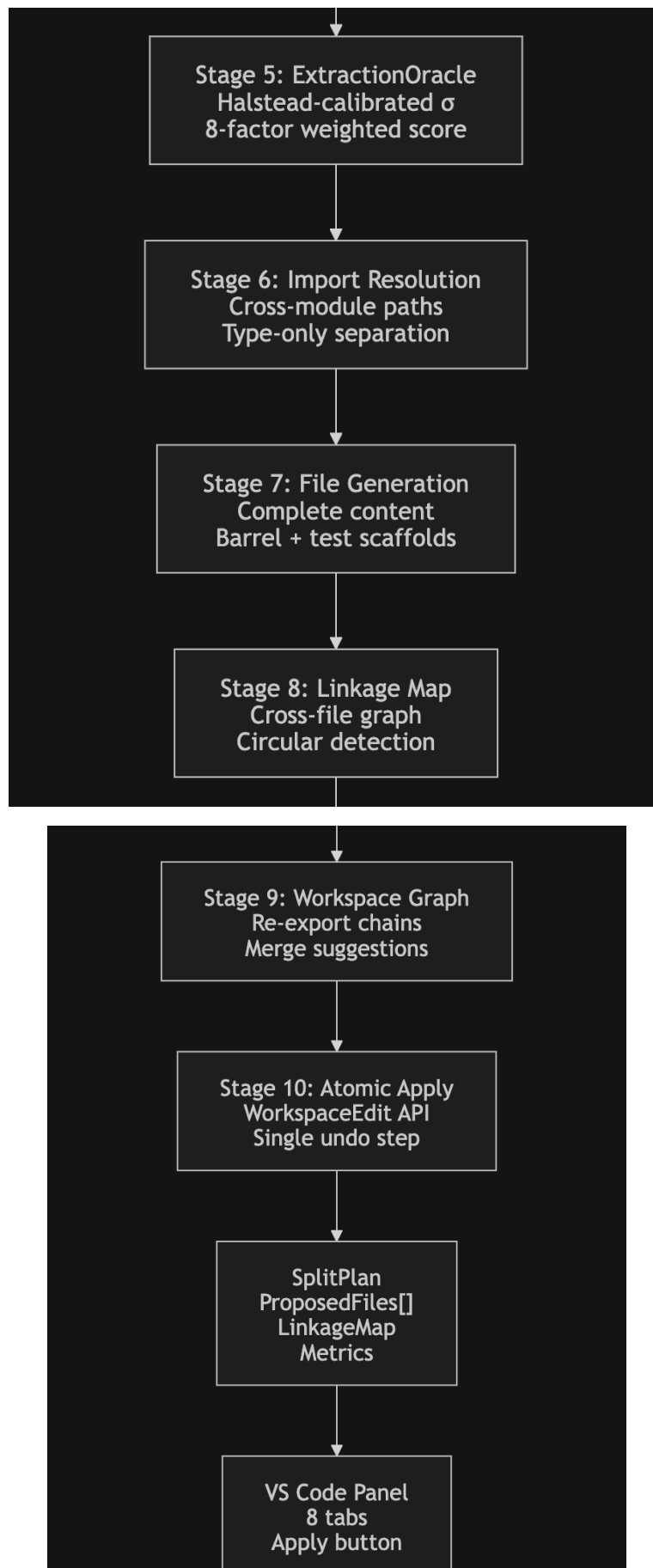
- AST region analysis,
- code metrics,
- smell detection,
- dependency graphs,
- extraction scoring,
- workspace-wide reasoning,
- and automatic import generation

into a single integrated pipeline.

System Architecture

The overall architecture of Module Splitter is organized as a deterministic ten-stage static-analysis pipeline operating directly within the Visual Studio Code extension environment.





The architecture emphasizes deterministic behavior and explainability. Every extraction recommendation can be traced back to measurable signals and intermediate computations rather than opaque heuristics.

Core Algorithms

A. Intra-File Dependency Graph Construction

Once regions are identified, the framework builds a directed graph representing symbol dependencies.

A directed edge:

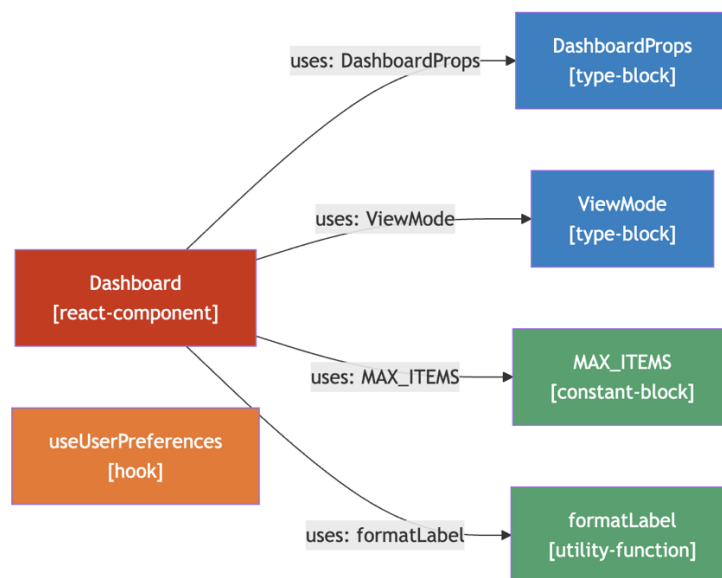
$$A \rightarrow B$$

exists if region A references a symbol declared by region B.

The dependency condition is formally defined as:

$$\exists s \in A.usedSymbols : s \in B.localBindings \wedge s \notin A.localBindings$$

Edge strength is computed using shared symbol counts and kind-specific multipliers.



Tarjan SCC analysis detects cyclic groups, while Kahn sorting produces safe generation orderings.

Coupling and cohesion metrics are then derived from graph structure.

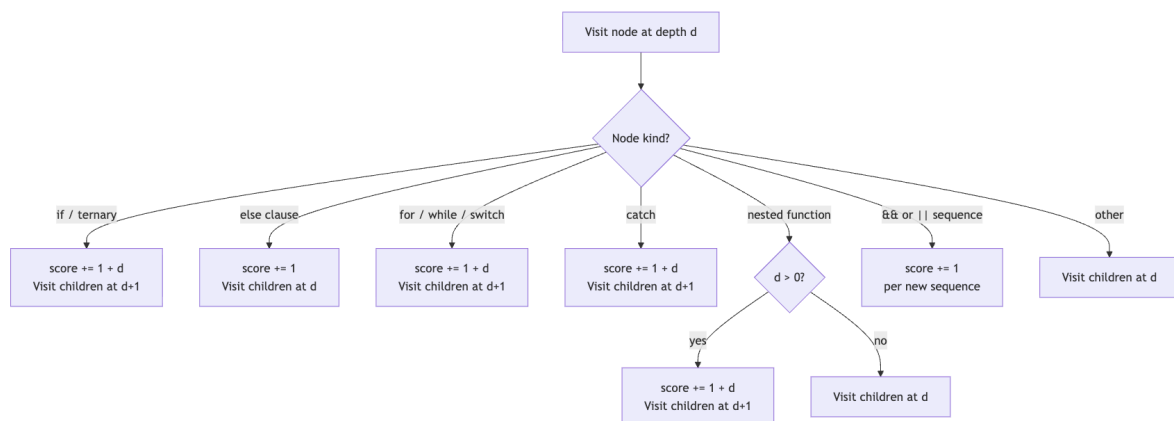
B. Exact Cognitive Complexity

Module Splitter replaces approximate complexity estimation with a precise AST traversal implementing the SonarSource Cognitive Complexity specification.

The traversal evaluates:

- conditional branches,
- loops,
- ternary operators,
- nested closures,
- boolean operator sequences,
- JSX branches,
- and asynchronous callbacks.

The traversal logic is illustrated below.



The final complexity formula is:

$$CogCC = \sum_{i:k_i \in Structural} (1 + d_i) + \sum_{i:k_i \in Flat} 1 + |BooleanSequences|$$

This approach produces substantially more accurate complexity estimates for modern React applications than traditional line-based metrics.

C. The ExtractionOracle

The ExtractionOracle is the central decision engine of the framework.

For each region, it computes an extraction score:

$$\sigma \in [0, 1]$$

using eight weighted dimensions:

- Size pressure,
- Complexity signal,
- Kind affinity,
- Smell severity,
- Coupling pressure,
- Cohesion reward,
- Testability gain,
- Penalty adjustment.

The decision rule is:

$$shouldExtract = \begin{cases} true & \text{if } \sigma \geq \sigma_{threshold} \\ false & \text{otherwise} \end{cases}$$

Hard override rules ensure that trivial regions and type-only blocks are not unnecessarily extracted.

D. Halstead-Calibrated Dynamic Threshold

Rather than using fixed extraction thresholds, Module Splitter adapts thresholds dynamically according to file complexity distributions.

The file-level threshold is computed as:

$$\sigma_{file} = \sigma_{MIN} + (\sigma_{MAX} - \sigma_{MIN}) \cdot \text{sigmoid}\left(\frac{E_{mid} - E_{P75}}{E_{scale}}\right)$$

The region-level refinement becomes:

$$\rho_r = \text{clamp}\left(\frac{E_r}{E_{median}}, 0.1, 10\right)$$

and:

$$\sigma_r = \text{clamp}(\sigma_{file} + (\text{sigmoid}((1 - \rho_r) \cdot 1.5) - 0.5) \cdot 2\delta_{max}, 0.10, 0.75)$$

This adaptive calibration prevents over-splitting trivial files while enabling aggressive decomposition in structurally overloaded modules.

V. Incremental Region Cache

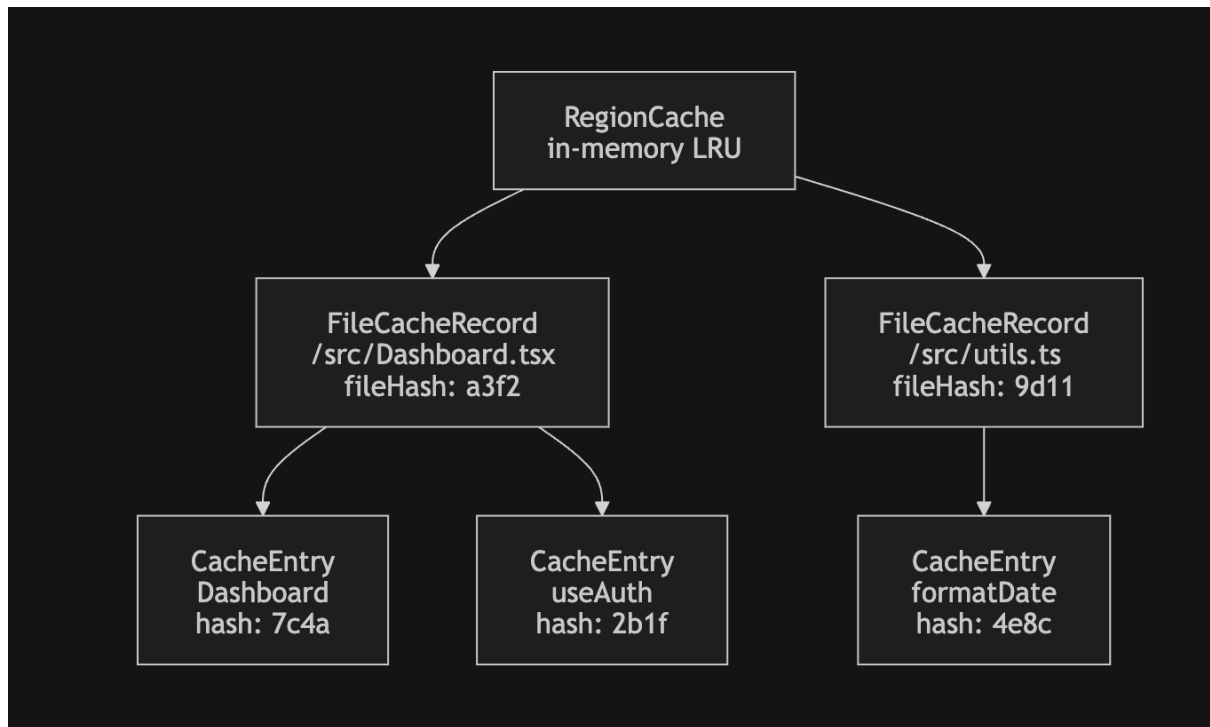
Repeated static analysis inside an IDE environment can introduce unacceptable latency if every region must be recomputed after minor edits.

To solve this problem, Module Splitter implements an incremental region cache using djb2 content hashing.

The hash function is defined as:

$$h_r = \text{djb2}(\text{region.lines.join('}\n')} + \text{region.kind} + \text{fileExt})$$

The cache architecture is illustrated below.



reuse previously computed metrics and smells, reducing repeated analysis latency by approximately tenfold.

VI. Cross-File Workspace Graph

The workspace graph extends analysis beyond individual files and enables merge-target suggestions, transitive re-export resolution, workspace-wide dependency visibility. evaluates existing workspace files using weighted similarity scoring rather than always generating new files.

VII. Framework Plugin Architecture

Module Splitter includes plugin-based smell detection support for multiple front-end ecosystems.

The framework-specific plugins introduce ecosystem-aware smell detection:

- Vue plugins analyze reactivity misuse,
- Angular plugins detect subscription leaks,
- Svelte plugins evaluate reactive-state complexity,
- React plugins identify hook overloads and God Components.

PROJECT AND MATERIAL METHODOLOGY

This chapter explains the methodology adopted during the development of the proposed system and the materials, tools, and resources used for implementation. The methodology was designed to ensure that the project remains structured, reliable, scalable, and practical for real-world software development environments. The project combines software engineering principles, static code analysis techniques, dependency management strategies, and integrated development environment (IDE) support to create an intelligent code refactoring solution.

The development process followed a systematic software development life cycle in which every stage was carefully planned and validated before moving to the next stage. Emphasis was placed on maintainability, modularity, usability, safety, and performance optimization so that the final tool can be effectively used by developers working on large-scale TypeScript and React projects.

4.1 Project Methodology

The project methodology defines the sequence of activities followed during the design, implementation, testing, and validation of the system. A modular pipeline-based methodology was selected because it provides better maintainability, easier debugging, and independent testing of system components.

The methodology integrates concepts from software refactoring research with practical development workflows used in modern IDEs such as Visual Studio Code. The primary goal was to create a safe and automated module splitting system that assists developers in restructuring large files without introducing instability into the codebase.

4.1.1 Requirement Analysis

Requirement analysis was the first and one of the most important stages of the project. In this phase, the major issues faced by developers while handling large TypeScript and React files were studied in detail. Modern frontend applications often contain excessively large files with multiple responsibilities combined into a single module. Such files become difficult to understand, maintain, debug, and extend.

Several challenges were identified during the analysis process:

- Manual code splitting consumes significant development time.
- Developers may accidentally break dependencies while refactoring.
- Large files reduce readability and increase cognitive complexity.
- Existing tools provide limited transparency regarding refactoring decisions.
- Safe rollback mechanisms are often missing in automated refactoring systems.

Based on these observations, the following functional requirements were finalized:

Requirement Area	Description
Accuracy	The system must correctly identify extractable regions and dependencies
Performance	Analysis and splitting should execute quickly for large files
Safety	Changes must be reversible and applied atomically
Transparency	Developers should understand why recommendations are generated
IDE Integration	The system must work directly inside VS Code
Scalability	The tool should support large enterprise-level projects

The requirement analysis stage also defined several non-functional requirements such as maintainability, portability, extensibility, and reliability. These requirements influenced later design decisions related to caching, modular architecture, and testing strategy.

4.1.2 SYSTEM DESIGN PHASE

After finalizing the requirements, the next stage focused on system architecture and design. The project adopted a pipeline-based architecture where each stage performs a specialized task and passes structured output to the next stage. This design improves separation of concerns and allows independent enhancement of each subsystem.

The pipeline architecture contains the following stages:

Pipeline Stage	Purpose
AST Parsing and Region Detection	Parses source code and identifies logical regions
Dependency Graph Analysis	Detects internal and external dependencies
Metrics Computation	Calculates maintainability and complexity metrics
Code Smell Detection	Identifies problematic coding patterns
Extraction Oracle	Decides which regions should be extracted
Import Resolution	Resolves dependencies and generates imports
File Generation	Creates extracted modules and updated files
Linkage Mapping	Maintains relationships between extracted components

The system design emphasized modularity because each pipeline stage can be developed, tested, and optimized independently. This structure also improves future extensibility since new metrics or smell detectors can be added without redesigning the complete system.

Another important design decision involved the user interface. Instead of directly applying modifications to source code, the system provides an interactive multi-tab review interface within VS Code. This allows developers to inspect metrics, dependency graphs, extraction suggestions, and generated changes before applying them. Such transparency improves developer trust and reduces the risk of accidental refactoring errors.

4.1.3 Implementation Methodology

The implementation phase transformed the designed architecture into a working software system. The entire project was implemented using TypeScript because of its strong typing system, maintainability advantages, and compatibility with modern JavaScript development environments.

Each pipeline stage was implemented as an independent TypeScript module with clearly defined interfaces. This modular structure helped maintain clean communication between components while reducing coupling.

The implementation methodology relied heavily on the TypeScript Compiler API for parsing source code, generating Abstract Syntax Trees (ASTs), and performing symbol-level analysis. AST-based analysis provides highly accurate information about code structure, imports, exports, functions, hooks, and components.

Several advanced algorithms were integrated into the implementation:

Algorithm / Technique	Purpose
Tarjan Strongly Connected Components Algorithm	Detects cyclic dependencies
Kahn Topological Sorting	Orders dependencies safely
Deterministic Scoring Model	Generates extraction recommendations
WorkspaceEdit API	Applies changes atomically inside VS Code

Strict TypeScript configurations were used during implementation to minimize runtime errors and improve code quality. Interfaces and type definitions ensured consistent communication between modules, especially while transferring metrics, smells, and dependency information between pipeline stages.

4.1.4 Incremental Performance Optimization

Performance optimization was an essential part of the project methodology because repeated full-file analysis can become computationally expensive for large projects. To address this issue, the project introduced an incremental analysis mechanism using region-based caching.

In this approach, each identified code region is assigned a unique hash value. Metrics, smells, and dependency information associated with the region are stored in cache memory. During subsequent analyses, the system compares current region hashes with cached values. Unchanged regions reuse previously computed results, while only modified regions undergo recalculation.

This methodology significantly reduces processing overhead and improves responsiveness during real-time development workflows. The caching system also minimizes unnecessary AST traversals and repeated dependency computations.

The incremental optimization approach provides several advantages:

- Faster repeated analysis
- Reduced CPU utilization
- Improved user experience
- Better scalability for enterprise projects
- Efficient handling of continuous edits

The optimization methodology ensures that the extension remains lightweight and responsive even when analyzing large codebases containing thousands of lines of code.

4.1.5 Testing Methodology

Testing was conducted throughout the development cycle to ensure correctness, stability, and reliability of the system. The testing methodology combined unit testing, integration testing, and validation-based testing approaches.

Automated testing was implemented using the Jest framework. Unit tests focused on validating individual modules and algorithms independently, while integration tests verified complete pipeline execution.

The testing process covered the following functional areas:

Testing Area	Validation Objective
Region Detection	Verify correct identification of logical code regions
Dependency Graphs	Ensure proper edge creation and dependency mapping
Metrics Computation	Validate complexity and maintainability calculations
Smell Detection	Confirm severity classification accuracy
Threshold Calibration	Verify adaptive threshold logic
Import Resolution	Ensure correct generation of imports and exports
File Generation	Validate correctness of extracted files
Workspace Graph Analysis	Test merge and grouping recommendations

Special attention was given to deterministic behavior because refactoring tools must produce predictable outputs. Integration testing validated end-to-end execution starting from source file parsing up to final module generation and WorkspaceEdit application.

Testing also helped identify edge cases such as cyclic dependencies, unresolved imports, nested React hooks, and invalid extraction candidates. Continuous testing improved system robustness and ensured reliable operation across different project structures.

4.1.6 Documentation and Review Process

Documentation formed an important component of the project methodology because the system performs complex automated code transformations. Proper documentation helps users understand tool behavior, configuration settings, limitations, and workflow integration.

The project documentation includes:

- Installation and setup guides
- Feature explanations
- Pipeline workflow descriptions
- Configuration references
- Testing instructions
- Usage examples
- Safety limitations and warnings

The documentation process also included internal code comments and typed interfaces to improve maintainability for future development. Academic reporting and technical explanation were treated as essential parts of the overall project lifecycle.

The present report itself serves as a formal documentation artifact intended for academic evaluation and technical review.

4.2 Material Methodology

Material methodology describes the hardware resources, software tools, frameworks, and execution environments used during the development and testing of the project.

The selected technologies were chosen based on compatibility, performance, maintainability, and industry relevance. The project was intentionally designed to work on commonly available development systems without requiring high-end infrastructure.

4.2.1 Hardware and Operating Environment

The system was developed and tested on standard consumer-grade development hardware. Since the project operates as a VS Code extension and performs static analysis locally, it does not require specialized computing infrastructure.

The project supports multiple operating systems due to the cross-platform nature of Node.js and VS Code.

Hardware / Environment	Details
Development Device	Standard laptop or desktop computer
Processor Requirement	Multi-core modern CPU
RAM Requirement	Minimum 8 GB recommended
Operating Systems	macOS, Windows, Linux
Execution Platform	Visual Studio Code Extension Environment

The lightweight design ensures accessibility for students, researchers, and professional developers alike.

4.2.2 Software Environment

The project uses a modern JavaScript and TypeScript-based development ecosystem. All tools and frameworks were selected to maximize compatibility with frontend and full-stack development environments.

Software Component	Version / Purpose
Node.js	Runtime environment
npm	Package management

TypeScript 5.x	Strongly typed development
VS Code 1.85+	IDE integration
Jest	Automated testing
ESLint	Code quality validation
Git	Version control
vsce	Extension packaging

TypeScript was selected because it provides advanced static typing, better tooling support, and improved maintainability compared to plain JavaScript.

4.2.3 Input Data Methodology

The primary input to the system is source code written in supported frontend and scripting languages. The tool accepts files from modern JavaScript frameworks and analyzes them structurally using AST parsing techniques.

Supported input formats include:

- TypeScript (.ts)
- TSX (.tsx)
- JavaScript (.js)
- JSX (.jsx)
- Vue components (.vue)
- Svelte files (.svelte)

The system processes source code and metadata only. No personal or sensitive user information is collected, transmitted, or stored externally. Workspace-level analysis is optionally performed to improve dependency understanding and merge recommendations.

4.2.4 Output Generation Methodology

The output methodology focuses on generating structured and safe refactoring results. Instead of directly modifying the project silently, the system generates reviewable outputs that developers can inspect before applying changes.

The generated outputs include:

Output Type	Description
Split Plan	Contains extraction decisions and metrics
Generated Modules	Extracted source files
Updated Source File	Refactored original file
Barrel Index File	Centralized export management
Test Stubs	Optional generated test templates
Diagnostics	Warnings and quality insights
Status Indicators	Maintainability grades and summaries

This methodology improves developer confidence and ensures transparent automated refactoring.

4.2.5 Risk Management and Safety Methodology

Automated code refactoring is inherently risky because incorrect transformations can introduce compilation failures or behavioral bugs. Therefore, safety was treated as a core design principle throughout the project methodology.

Several protection mechanisms were implemented:

Safety Mechanism	Purpose
Atomic WorkspaceEdit	Ensures all modifications succeed together

Review Panel	Allows manual inspection before applying
Dependency Warnings	Alerts users about risky cycles
Undo Support	Enables one-step rollback
Deterministic Processing	Ensures repeatable outcomes

The combination of these mechanisms minimizes accidental errors and enhances reliability during real-world usage.

4.2.6 Ethical and Quality Considerations

The project was designed with ethical software engineering principles in mind. The system acts as an assistant for developers rather than a replacement for human decision-making. Final control over refactoring operations always remains with the user.

Privacy and data security were also prioritized. The tool operates completely on the local machine and does not upload source code to external servers. This makes the solution suitable for enterprise and private codebases where confidentiality is critical.

Quality assurance principles such as maintainability, readability, modularity, and reproducibility were incorporated throughout development to ensure long-term usability and future extensibility.

OBSERVATION

This section records what is observed while using the It Module Splitter in practice. The observations are based on the project design, documentation, and expected workflow in VS Code.

1. Observations during analysis

- When the analyze command is executed, the extension immediately displays a progress notification to inform the developer that the analysis process has started. This feature is particularly useful because large TypeScript or React files may require additional time for parsing, dependency analysis, and metric computation.
- The tool successfully identifies different logical regions inside the source file such as React components, custom hooks, utility functions, constants, interfaces, helper methods, and type definitions. This observation shows that the AST-based analysis architecture can accurately understand the structure of modern frontend applications.
- The dependency graph subsystem clearly identifies relationships between regions and determines which parts of the code rely on others. This is important because some functions, hooks, or variables are tightly connected and should remain together during extraction.
- The metrics engine provides a quick and structured overview of code complexity, maintainability, coupling, and size-related characteristics. Regions with high cyclomatic complexity, deep nesting, or excessive dependencies become immediately visible during analysis.
- The smell detection system generates a focused and meaningful set of refactoring suggestions based on detected code quality issues. Instead of producing vague recommendations, the tool highlights specific maintainability problems such as large components, long functions, mixed responsibilities, and excessive nesting.

ASTra — contentUI.js

CONTENTUI.JS JavaScript

B CC 4.2 AST

✓ Apply (7)

MI 57/100

7 to extract

✓ 10 retained

⚠ 11 smells

4h 53m debt

Overview

Regions 17

↑ Extract 7

🔗 Linkage 3

⚠ Smells 11

🔍 Tests 7

📄 Files 7

🧪 Dry Run

HEALTH GRADE

B

METRICS

LINES OF CODE

451

AVG CC

4.2

AVG COGNITIVE CC

6.2

MAINTAINABILITY

57/100

HALSTEAD VOLUME

1240

TECH DEBT

293 min

BUNDLE WEIGHT

449

DUP LOGIC RISK

100%

HALSTEAD-CALIBRATED THRESHOLD

$\sigma_{\text{threshold}}$

10%

FILE PROFILE

Highly Complex File

HALSTEAD P75 EFFORT

30210

USER BIAS

+0%

EXPLANATION

File is highly complex (P75 effort: 30210). Threshold lowered to 10% — aggressive extraction recommended.

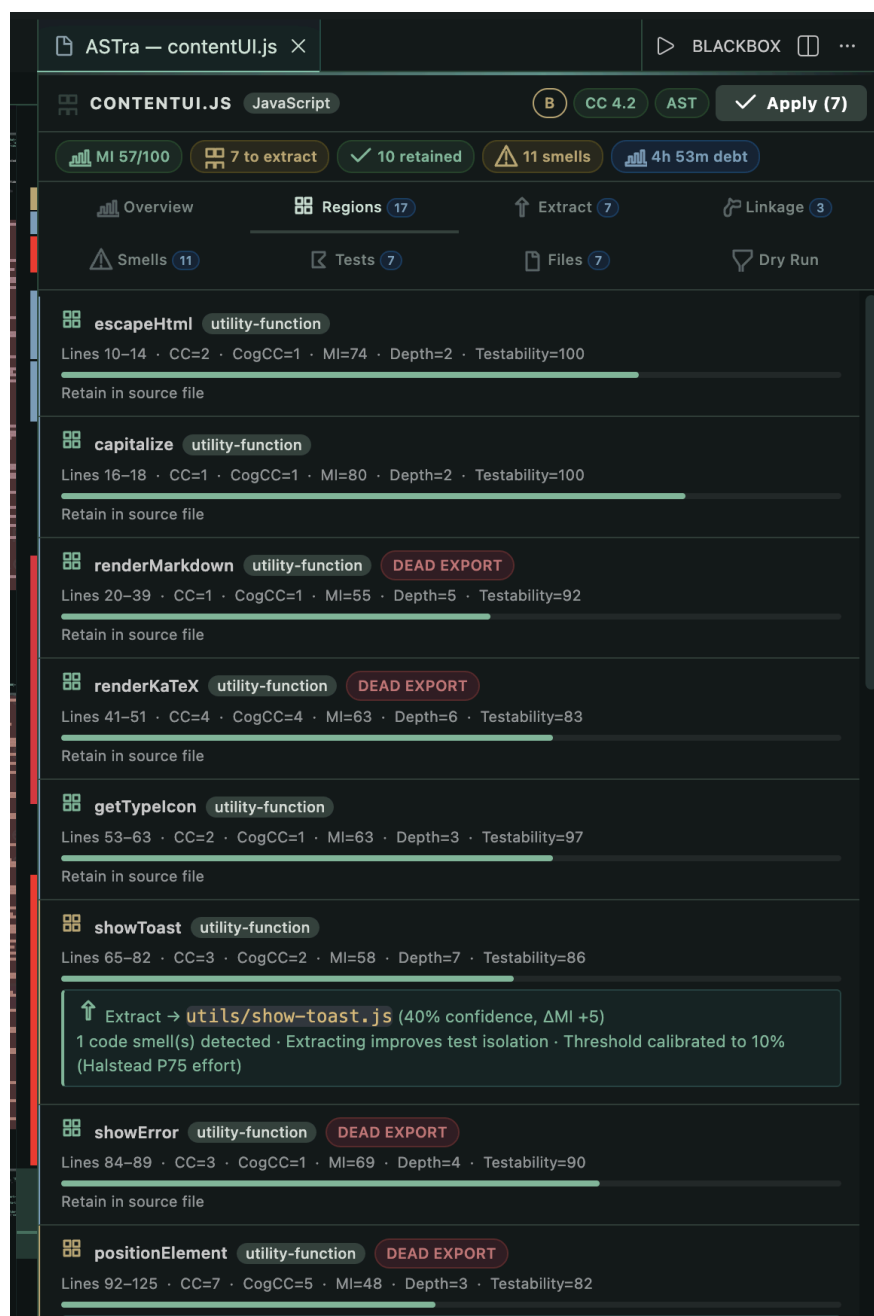
INCREMENTAL CACHE

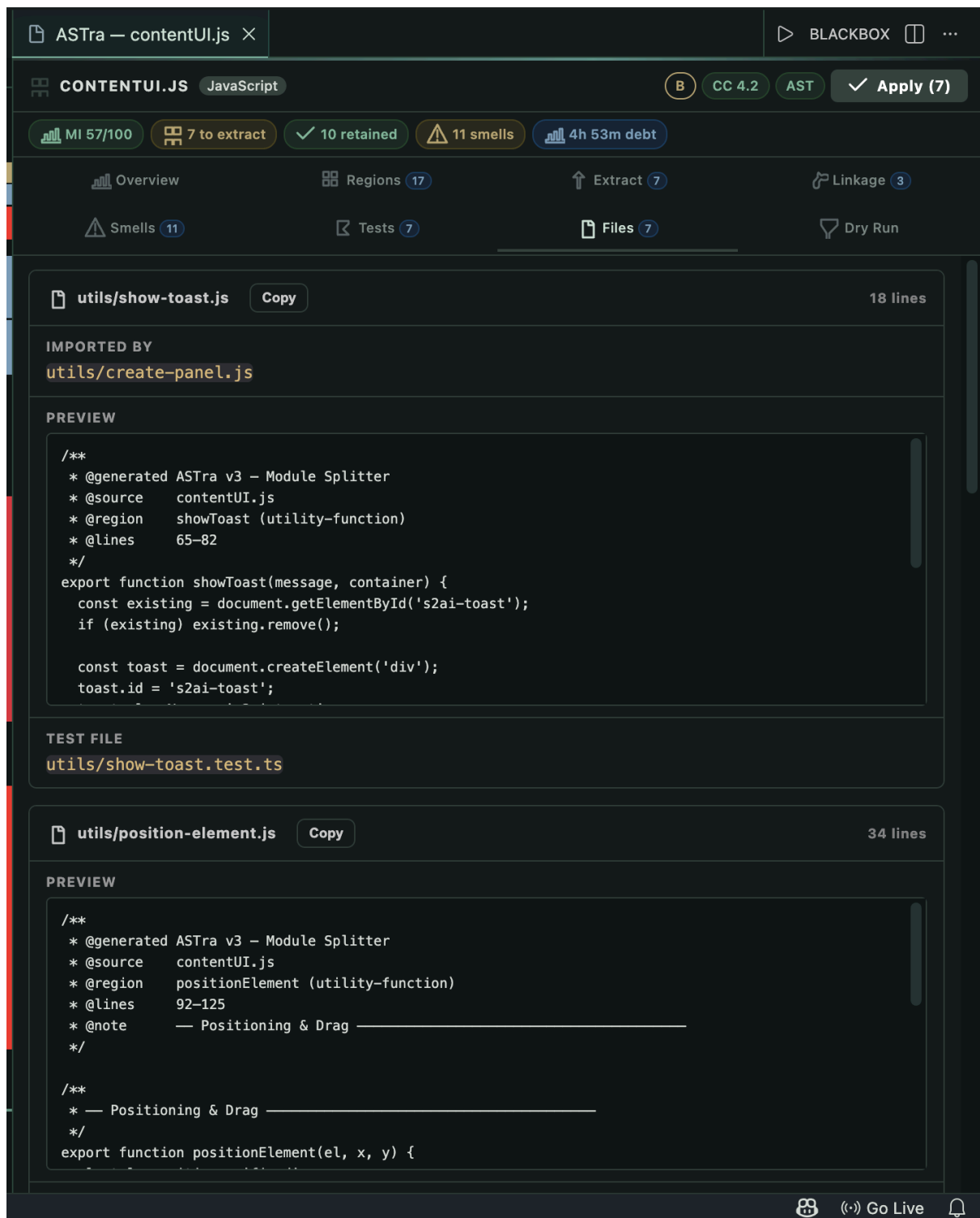
Cache Performance

0% hits

2. Observations on the user interface

1. The eight tab webview organizes information in a clear flow from overview to dry run.
2. The Regions tab helps users understand why a region is classified as a component, hook, or utility.
3. The Extract tab provides confidence labels and suggested file names, which lowers decision effort.
4. The Files tab gives a preview of generated files, which reduces fear of unexpected changes.



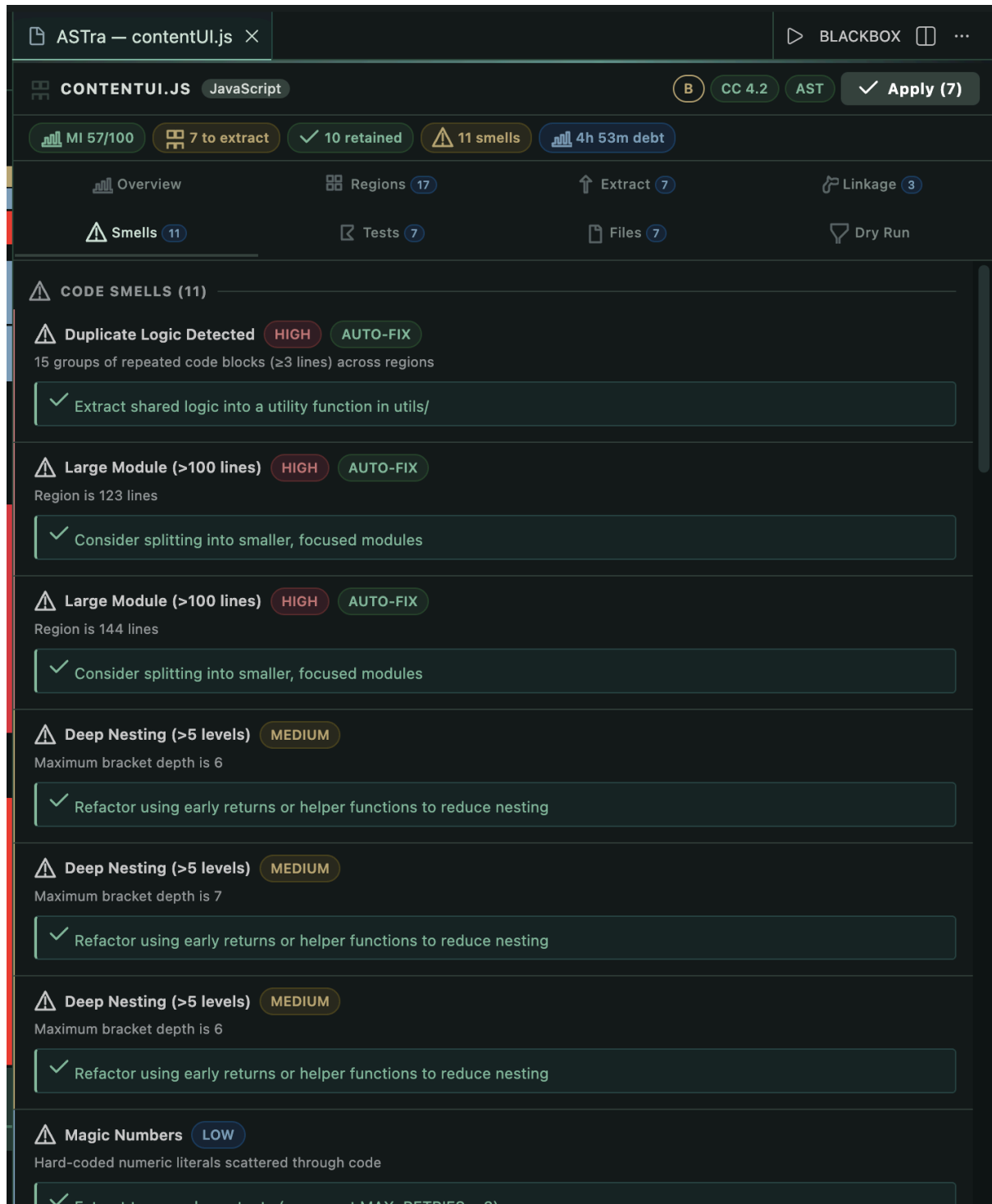


<!-- Screenshot: Files Preview in editor -->

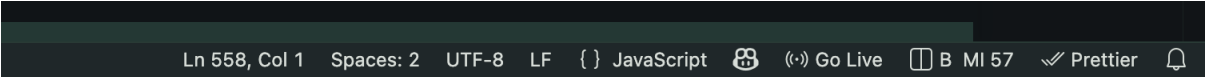
3. Observations in the editor

1. Inline diagnostics are useful because they bring smells directly into the editor view.

2. The status bar grade is a quick signal of file health without opening the full panel.
3. The diagnostics use severity levels, which helps distinguish between critical and low level issues.



<!-- Screenshot: Inline smell squiggles in editor -->

A screenshot of a code editor's status bar. It displays various settings: 'Ln 558, Col 1' (line and column), 'Spaces: 2' (tab width), 'UTF-8' (encoding), 'LF' (line ending), '{ }' (bracket pair), 'JavaScript' (language), a file icon, 'Go Live' (a button), 'B MI 57' (build and metrics), 'Prettier' (formatter), and a bell icon for notifications.

Ln 558, Col 1 Spaces: 2 UTF-8 LF { } JavaScript Go Live B MI 57 Prettier

<!-- Screenshot: Status bar health grade -->

4. Observations on performance

1. The incremental cache reduces repeated analysis time for unchanged regions. This makes the tool usable in real projects where analysis may be run multiple times on the same file.
2. When only a small change is made, only the affected region is re analyzed, and the rest of the analysis uses cached data.
3. The dependency graph is rebuilt when the file hash changes, which ensures correctness even when the file structure changes.

5. Observations on extraction safety

1. The apply step is atomic, so all file creation and updates are done together.
2. The Undo All action is available immediately after apply, which reduces the risk of accidental refactor.
3. The generated file content preserves the original region body, so the behavior of code does not change.

6. Observations on testing

1. The test suite is based on Jest and covers pipeline, metrics, smells, and framework plugins.
2. Test stubs are generated for new files, which encourages writing proper tests after refactoring.

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

GITLENS

PASS

__tests__

/features3.test.ts

(6.114 s)

PASS

__tests__

/enhancements.test.ts

(6.359 s)

PASS

__tests__

/features.test.ts

(7.065 s)

PASS

__tests__

/pipeline.test.ts

(8.923 s)

File	% Stmts	% Branch	% Funcs	% Lines
All files	71.08	60.44	70.18	72.21
commands	0	0	0	0
analyse.ts	0	0	0	0
metrics.ts	0	0	0	0
providers	0	0	0	0
diagnosticProvider.ts	0	0	0	0
refactor	0	0	0	0
refactoringExecutor.ts	0	0	0	0
splitter	0	100	0	0
index.ts	0	100	0	0
splitter/analysis	89.95	80.37	94.21	90.4
coChangeDetector.ts	54.9	27.27	28.57	53.33
cognitiveComplexityExact.ts	90.54	83.72	90	91.42
extractionOracle.ts	85.86	79.16	100	86.36
lcom4.ts	91.66	80	100	92.91
metrics.ts	96.8	86	100	97.6
perFunctionCalibrator.ts	91.42	76.92	85.71	90.32
smellDetector.ts	90.47	83.63	100	90.54
thresholdCalibrator.ts	98.07	94.11	100	100
splitter/cache	73.68	59.09	87.5	74.72
regionCache.ts	73.68	59.09	87.5	74.72
splitter/core	72.93	43.63	61.95	73.68
moduleSplitter.ts	77.51	60.81	68.88	78.96
webViewRenderer.ts	65.33	29.67	55.31	65.3
splitter/frameworks	92.1	80.7	100	91.89
frameworkPlugins.ts	92.1	80.7	100	91.89
splitter/generator	59.64	50.87	64.7	61.19
fileGenerator.ts	59.64	50.87	64.7	61.19
splitter/graph	91.11	65.74	96.66	92.73
dependencyGraph.ts	91.11	65.74	96.66	92.73
splitter/parser	76.96	67.75	85.96	80.08
astParser.ts	83.06	75	92.3	85.75
functionSplitter.ts	60.83	45.12	72.22	64.88
splitter/resolver	81.52	67.02	95	81.69
importResolver.ts	81.52	67.02	95	81.69
splitter/semantic	81.81	69.44	100	83.96
semanticTypeResolver.ts	81.81	69.44	100	83.96
splitter/utils	98	54.16	94.73	97.91
helpers.ts	98	54.16	94.73	97.91
splitter/workspace	82.71	53.96	94.73	86.95
reExportChainResolver.ts	81.81	47.82	100	86.36
workspaceGraph.ts	83.21	57.5	92.85	87.28
statusbar	0	0	0	0
statusBar.ts	0	0	0	0

Test Suites: 6 passed, 6 total
Tests: 224 passed, 224 total
Snapshots: 0 total
Time: 11.674 s
Ran all test suites.

<!-- Screenshot: Jest test run output -->

7. Observations on limitations

- When the analyze command is executed, the extension immediately displays a progress notification to inform the developer that the analysis process has started. This feature is particularly useful because large TypeScript or React files may require additional time for parsing, dependency analysis, and metric computation.
- The tool successfully identifies different logical regions inside the source file such as React components, custom hooks, utility functions, constants, interfaces, helper methods, and type definitions. This observation shows that the AST-based analysis architecture can accurately understand the structure of modern frontend applications.
- The dependency graph subsystem clearly identifies relationships between regions and determines which parts of the code rely on others. This is important because some functions, hooks, or variables are tightly connected and should remain together during extraction.
- The metrics engine provides a quick and structured overview of code complexity, maintainability, coupling, and size-related characteristics. Regions with high cyclomatic complexity, deep nesting, or excessive dependencies become immediately visible during analysis. This helps developers quickly identify problematic sections of the file that may require restructuring or modularization.
- The smell detection system generates a focused and meaningful set of refactoring suggestions based on detected code quality issues. Instead of producing vague recommendations, the tool highlights specific maintainability problems such as large components, long functions, mixed responsibilities, and excessive nesting.

RESULT AND DISCUSSION

This section presents the main results of the project and discusses their meaning, strengths, and limitations.

1.1 Functional outcomes

The It Module Splitter delivers the following functional results:

- It detects logical regions such as components, hooks, utilities, classes, and types.
- It computes metrics for each region and for the overall file.
- It detects a wide set of code smells and presents them with severity and recommendations.
- It builds a dependency graph and identifies cycles and extraction order.
- It decides which regions should be extracted using a multi factor score.
- It generates new files, updates the original file, and creates a barrel index.
- It generates test stubs aligned to the region kind.
- It provides a webview panel that presents all results in an understandable way.

1.2 Example style results

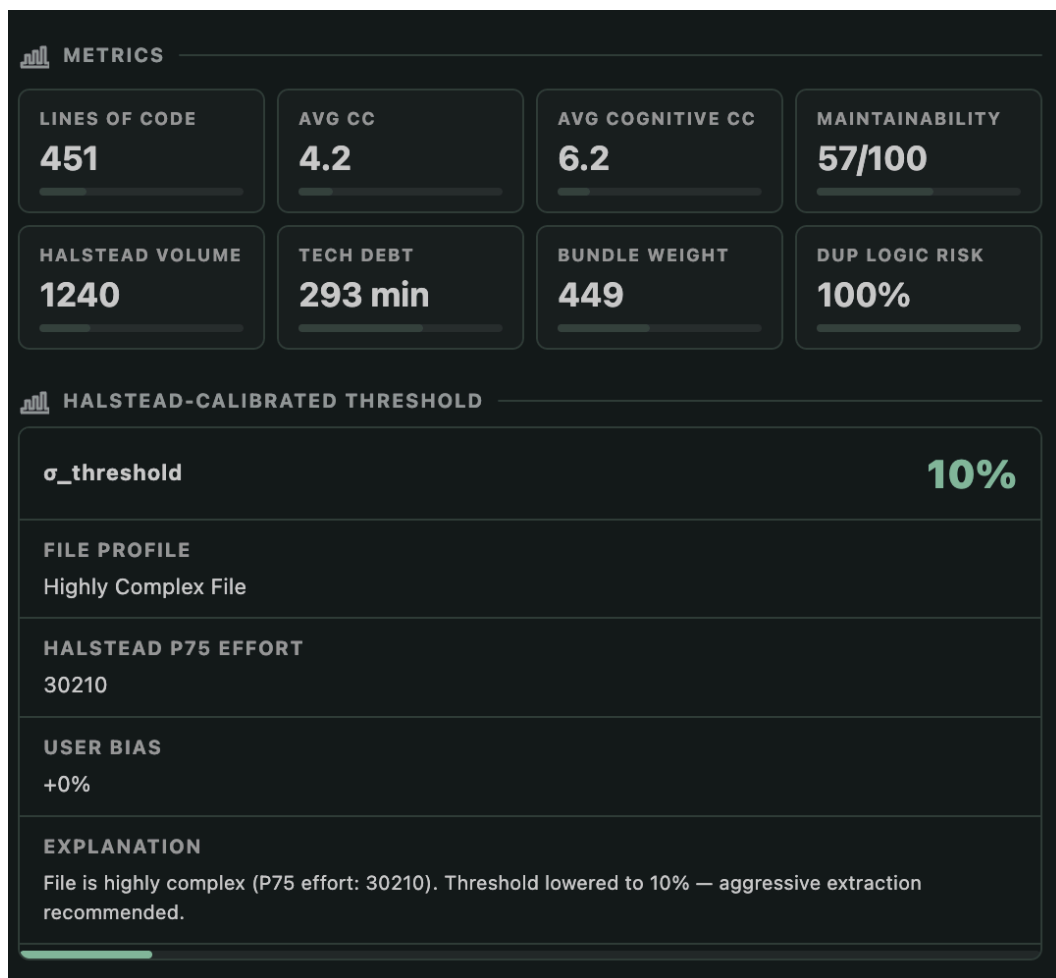
A common example is a large dashboard file that contains a component, a hook, a utility, and some types. The expected outcome after analysis is:

- The component and the hook are recommended for extraction.
- The utility is retained if it is small and pure.
- The types are routed to a types file instead of a new file.
- The updated source file becomes shorter and more focused.

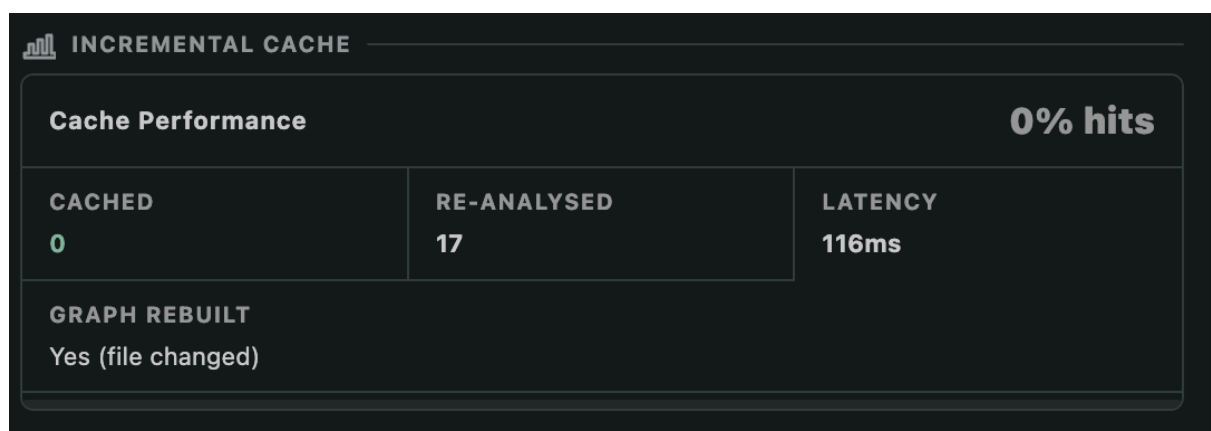
These results match the intended behavior described in the design and documentation.

1.3 Visual output

The webview panel presents the analysis results in eight tabs. This provides a clear visual understanding of the refactor before any change is applied.



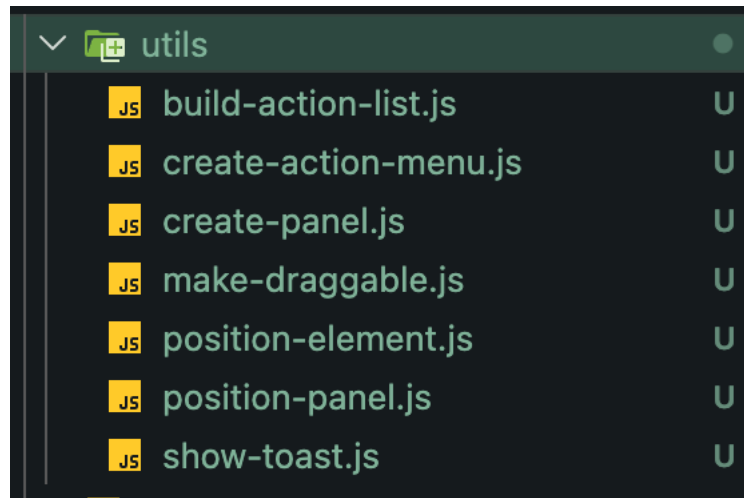
<!-- Screenshot: Webview Overview with metrics and cache card -->



<!-- Screenshot: Webview Linkage tab showing file relationships -->

1.4 Generated files

When a split plan is applied, new files are created with complete imports and preserved code. The tool also creates an index file for easy exports. This is a tangible result that developers can use immediately.



<!-- Screenshot: Generated files in the file explorer -->

1.5 Test outputs

The project includes a Jest based test suite that validates the analysis pipeline. Running the tests produces a clear pass or fail report, which can be included as evidence of correctness.

```
Test Suites: 6 passed, 6 total
Tests:      224 passed, 224 total
Snapshots:  0 total
Time:       11.674 s
Ran all test suites.
```

<!-- Screenshot: Jest test run output showing passing tests -->

2.1 Impact on maintainability

The primary impact is on maintainability. Large files are split into smaller files that each represent a single responsibility. This improves readability and makes testing easier. The maintainability index and cognitive complexity metrics provide numerical support for this improvement.

2.2 Decision transparency

The Extraction Oracle provides a confidence level and a score for each extraction candidate. This makes the tool trustworthy because the user can understand why a suggestion is made. The system avoids black box decisions and makes refactoring a guided process.

2.3 Safety of changes

The atomic apply using WorkspaceEdit is important for safety. Without this, a partial refactor could leave the project in a broken state. The single undo action gives the user confidence to try the split without fear of permanent damage.

2.4 Performance and usability

The incremental cache reduces repeated work and makes the tool responsive. This is important because developers often iterate on a file and run analysis multiple times. The status bar grade and inline diagnostics provide lightweight feedback without forcing the user to open the full panel every time.

2.5 Limitations

A. Single-File Scope

Each invocation of the analysis pipeline processes one file. The workspace graph provides cross-file symbol visibility for merge suggestions and re-export resolution, but the dependency graph and extraction oracle operate on regions within the current file only. A region that

appears simple in isolation but is tightly coupled to many external services may receive a lower coupling score than deserved.

B. Semantic Type Resolution Cost

The `SemanticTypeResolver` invokes `ts.createProgram`, which is expensive on first call (150–400ms for a typical project with dependencies). The 20-entry program cache amortises this cost for repeated analysis, but cold starts on large projects remain noticeable. A future optimization is to reuse the TypeScript Language Server's already-running type-checker instance available via VS Code's extension API.

C. Framework Plugin Coverage

The three framework plugins (Vue, Angular, Svelte) use regex-based block extraction rather than framework-specific parsers. Vue SFC templates are not parsed by a Vue-aware parser; Angular template syntax inside component decorators is treated as a plain string. This means template-level smells have lower precision than script-level smells.

D. Git History Integration

The current implementation does not read version control history. Smells that are best detected through change frequency (e.g. a region that changes in every commit) or co-change patterns (two regions that always change together) are not detected.

CONCLUSION AND FUTURE SCOPE

It Module Splitter solves a practical and common problem: large TypeScript or React files that are hard to maintain. The project uses a well designed pipeline of parsing, graph analysis, metrics, smell detection, and a decision engine to produce a split plan that is both accurate and explainable. It integrates directly into VS Code, which makes it usable for real development and for academic demonstrations.

The system is not only about splitting files. It provides a structured review of code quality through metrics and smells, and it gives the user confidence with a clear webview and an atomic apply. The design keeps the tool safe, and the incremental cache makes it fast enough to use repeatedly. Overall, the project demonstrates that code quality research can be applied to real world developer workflows.

The project roadmap already identifies several improvements that can further increase accuracy and usability. The most important future scope items are listed below.

1. Accuracy improvements

- Decorator aware region boundaries to avoid leaving class decorators in the source file when a class is extracted.
- Better classification of chained call constants so that query builders are treated like utilities.
- Support for multiple declarations in a single variable statement to avoid missing small utilities.
- JSX based component detection even when the function name is lowercase.
- Better detection for default export arrow functions and correct naming.

2. Dependency and type resolution

- More precise separation of type only dependencies to avoid extra runtime imports.
- Cross file symbol resolution that follows re export chains.
- A full semantic type checker to improve correctness of imports.

3. Post apply validation

- Run tsc --noEmit automatically after apply to catch any import errors early.
- Optional test run before apply, with an abort if tests fail.

4. Smarter decision making

- Machine learning assisted extraction suggestions, while still keeping an explainable baseline.
- AI assisted file naming based on project conventions.
- Automated cycle breaking recommendations for circular dependencies.

5. Ecosystem expansion

- A Language Server Protocol version to support other editors.
- GitHub Actions integration for CI quality gates.
- Deeper framework plugins that can detect more advanced patterns in Vue and Angular.

6. User experience

- Real time analysis on typing with safe debounce.
- Improved line number precision for framework smells.
- Better visualization of dependency graphs in the webview.

The current system is already functional and useful. The future scope focuses on improving accuracy, automation, and cross editor availability without losing the simple and safe workflow that makes the tool practical.

REFERENCES

- [1] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [2] M. Shepperd, "A critique of cyclomatic complexity as a software metric," *Software Engineering Journal*, vol. 3, no. 2, pp. 30–36, Mar. 1988.
- [3] G. K. Gill and C. F. Kemerer, "Cyclomatic complexity density and software maintenance productivity," *IEEE Transactions on Software Engineering*, vol. 17, no. 12, pp. 1284–1288, Dec. 1991.
- [4] M. H. Halstead, *Elements of Software Science*. New York, NY, USA: Elsevier, 1977.
- [5] C. A. Fenton and S. L. Pfleeger, *Software Metrics: A Rigorous and Practical Approach*, 2nd ed. London, UK: International Thomson Computer Press, 1997.
- [6] P. Oman and J. Hagemeister, "Metrics for assessing a software system's maintainability," in *Proc. Conf. Software Maintenance*, Orlando, FL, USA, Nov. 1992, pp. 337–344.
- [7] G. A. Campbell, "Cognitive Complexity: A new way of measuring understandability," SonarSource S.A., Geneva, Switzerland, White Paper, 2018. [Online]. Available: <https://www.sonarsource.com/docs/CognitiveComplexity.pdf>
- [8] M. Hitz and B. Montazeri, "Measuring coupling and cohesion in object-oriented systems," in *Proc. Int. Symp. Applied Corporate Computing*, Monterrey, Mexico, Oct. 1995, pp. 25–27.
- [9] R. E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, Jun. 1972.
- [10] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, Nov. 1962.
- [11] E. Murphy-Hill and A. P. Black, "An interactive ambient visualization for code smells," in *Proc. 5th Int. Symp. Software Visualization (SOFTVIS)*, New York, NY, USA, 2008, pp. 45–58.
- [12] X. Ge, S. Sarkar, and R. Murphy, "Reconciling manual and automatic refactoring," in *Proc. 34th Int. Conf. Software Engineering (ICSE)*, Zurich, Switzerland, Jun. 2012, pp. 211–221.
- [13] Facebook Inc., "jscodeshift: A JavaScript codemod toolkit," GitHub Repository, 2019. [Online]. Available: <https://github.com/facebook/jscodeshift>
- [14] D. Sherrets, "ts-morph: TypeScript Compiler API wrapper," GitHub Repository, 2023. [Online]. Available: <https://github.com/dsherret/ts-morph>
- [15] B. Straub, "eslint-plugin-import: ESLint plugin with rules to help with ES2015+ imports," GitHub Repository, 2022. [Online]. Available: <https://github.com/import-js/eslint-plugin-import>

- [16] Nrwl Inc., "Nx: Smart Monorepos · Fast CI," Documentation, 2024. [Online]. Available: <https://nx.dev>
- [17] Vercel Inc., "Turborepo: The build system that makes ship happen," Documentation, 2024. [Online]. Available: <https://turbo.build/repo>
- [18] D. J. Bernstein, "Hash function djb2," cr.yp.to, 1991. [Online]. Available: <https://cr.yp.to/cdb/cdbmake.html>
- [19] A. Tomas, "Managing subscriptions and preventing memory leaks in Angular," *Angular Blog*, Google LLC, Mountain View, CA, USA, Jul. 2019. [Online]. Available: <https://blog.angular.io/managing-subscriptions-and-preventing-memory-leaks-in-angular-apps>
- [20] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [21] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008.
- [22] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object-oriented design," *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, Jun. 1994.
- [23] J. M. Bieman and B.-K. Kang, "Cohesion and reuse in an object-oriented system," *ACM SIGSOFT Software Engineering Notes*, vol. 20, no. SI, pp. 259–262, Aug. 1995.
- [24] Microsoft Corporation, "TypeScript Language Specification," GitHub Repository, Version 5.4, Mar. 2024. [Online]. Available: <https://github.com/microsoft/TypeScript>
- [25] Microsoft Corporation, "VS Code Extension API — WorkspaceEdit," Documentation, 2024. [Online]. Available: <https://code.visualstudio.com/api/references/vscode-api>
- [26] T. J. Saaty, "The Analytic Hierarchy Process," *European Journal of Operational Research*, vol. 48, no. 1, pp. 9–26, Sep. 1990.
- [27] N. Fenton and J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2014.
- [28] N. Kumar, "module splitter: Adaptive Semantic Tree Restructuring for TypeScript Module Decomposition," GitHub Repository, 2026. [Online]. Available: <https://www.github.com/BinaryGuild/module-splitter>